GovMut-Health: A Mutation Benchmark for Sequence-Sensitive Governance Faults in Stateful Health-AI Systems

Nguyen Ngoc Thien and Trinh Minh Quang

Hai Ba Trung High School, Hue, Vietnam
kakangocthien109@gmail.com

Abstract. Stateful health-AI governance failures often depend on a sequence of operations rather than a single input. A test may pass when a disclosure is created, consent is valid, and a proposal is formed, yet still miss a defect that appears only after consent changes, a stale proposal is retried, or cached state is reused before persistence. GovMut-Health evaluates how well established testing strategies detect these sequence-sensitive defects. The contribution is not mutation testing, property-based testing, or state-machine testing themselves; it is an executable health-AI governance fault model, a reviewed mutant corpus mapped to contract invariants, and a mutant-level comparison of four frozen strategies. The sealed experiment evaluated 45 non-equivalent mutants across 16 method/seed slots, producing 720 executions with zero infrastructure errors. Mutation scores were 35.6% for regression tests, 8.9% for stateless properties, 13.3% for state-machine testing, and 44.4% for the combined strategy. The combined strategy detected all 16 mutants killed by regression and four additional mutants, but that gain was not significant in the exact paired comparison ($p = 0.125$). Its gains over stateless properties and state-machine testing were significant. The results therefore support complementarity between conventional regression and generated governance tests on this frozen corpus, not a claim that one testing paradigm universally dominates or that the system is formally verified.

1 Introduction

A stateful governance contract can fail even when every individual API call appears reasonable. The defect may require a sequence: disclose data, change consent, reuse cached state, retry a rejected proposal, and then attempt persistence. Traditional example-based tests often cover important happy paths and known regressions, but they do not necessarily explore the orderings in which governance bugs emerge.

While general mutation testing evaluates syntactic bug detection (SWE-Mutation [9]) and formal agent verification tests idealized state machines (MemTX [7]), neither captures sequence-sensitive governance failures in stateful health architectures. GovMut-Health establishes the first domain-specific

mutation benchmark for healthcare AI, mapping 15 multi-operation fault families directly to health data lifecycle invariants to rigorously benchmark the fault-detection adequacy of regression, property-based, and state-machine testing strategies.

2 Background and Novelty Boundary

2.1 Model-based, property-based, and access-control testing

QuickCheck established property-based random testing, and model-based testing has long used explicit behavioral models to generate and assess traces [1,2]. Mutation testing evaluates test adequacy by seeding controlled faults and measuring which faults the tests detect [3]. Access-control research has likewise used model-based generation and policy mutation to test authorization systems [4,5,6]. None of these techniques is a GovMut-Health novelty.

2.2 Stateful agent assurance and SWE-Mutation

Recent agent-memory systems use property-based testing and bounded state exploration to check transactional invariants, while SWE-Mutation evaluates LLM-generated test suites on systematically mutated programs [7,9]. Work on bitemporal memory and concurrent agents also makes sequence-dependent state errors explicit [10,8]. However, SWE-Mutation targets generic code syntax mutations, whereas GovMut-Health introduces a domain-specific fault taxonomy for health data governance (e.g., dropping bitemporal conflict markers, stale cache reconstruction, downgrade leaks).

2.3 Surviving contribution

GovMut-Health contributes a domain-specific, executable mutation model for governance failures whose observable effect may require several operations. The benchmark maps each mutant to health-AI contract invariants such as state freshness, consent and policy freshness, subject and purpose binding, snapshot integrity, retry semantics, provenance, atomicity, and audit reconstruction. It then compares established testing strategies at the mutant level under a frozen budget. The novelty is the fault model and evaluation object, not the underlying testing methods.

3 Reference State Model

The abstract state is

$$X = (u, v_s, v_p, v_c, c, a, p, H, P, L, A), \tag{1}$$

Table 1. Core governance invariants fixed before the mutation experiment.

ID	Invariant
P1	Idempotent replay cannot duplicate committed persistent state.
P2	A stale base version cannot overwrite a newer state.
P3	A policy-version change invalidates a proposal that depends on the old policy.
P4	Consent revocation invalidates affected disclosure/write authorization.
P5–P7	Subject, actor, and purpose bindings cannot silently cross identities.
P8	Expired or digest-invalid snapshots cannot authorize a commit.
P9	A superseded assertion is absent from the next current-state disclosure.
P10	Temporal reconstruction preserves historical truth and explicit conflict.
P11	Provenance closure holds for committed assertions.
P12	Canonical ledger state survives deletion/rebuild of derived snapshots.
P13	Failed commit is atomic: no partial persistent state or corrupt audit state.
P14	Successful commit remains reconstructable from the audit/provenance path.
P15	Retry cannot turn a stale/governance-invalid proposal into unauthorized success.

where u is subject, v_s state version, v_p policy version, v_c consent version, c consent status, a actor, p purpose, H issued snapshots, P proposals, L canonical ledger state, and A audit evidence.

The model exposes operations for creating/superseding assertions, issuing a task-bounded snapshot, changing consent/policy, changing actor/purpose, proposing and committing writes, retrying conflicts, expiring/replaying snapshots, reconstructing historical state, and deleting/rebuilding derived snapshot/cache rows. Preconditions constrain operations to meaningful states; postconditions compare implementation observations with the reference model.

4 Testing Strategies

We compare four strategies on the same implementation revision. **M0** is the existing hand-authored regression/unit/integration suite. **M1** adds stateless property tests that vary values without maintaining a long operation history. **M2** uses an executable rule-based state machine that generates and shrinks sequences. **M3** combines regression, stateless property, and state-machine tests. If stable, an optional API-bound adapter replays selected state-machine traces through the deployed service and is reported separately from in-process testing.

The state-machine generator is biased toward transitions that create meaningful precondition changes rather than uniformly sampling invalid actions. Examples include issuing a snapshot immediately before a consent/policy change, generating competing writers, replaying a proposal after supersession, and rebuilding derived state before a stale replay. On failure, the framework stores the minimal shrunk sequence together with the seed and implementation revision.

5 Controlled Mutation Benchmark

The mutation manifest is frozen before the headline run. Each mutant is a test-only, reversible change representing one realistic fault. Mutants are not runtime feature flags available to production deployments. The 15 rows in Table 3 are

Table 2. Cross-Paradigm Mutation and Verification Benchmark Comparison.

Paradigm	State Invariants	Lineage	Trace	Bitemporal Math	Merkle Root	Target Domain
Syntactic Mutants (MutPy)	×	×	×	×		Generic Code
MemTxn / Cordon [?,?]	✓	×	×	×		Agent Memory
TOKI Algebra [10]	✓	×	✓	×		Bitemporal Memory
FHIR Version OCC [13]	✓	×	×	×		EHR REST
GovMut-Health (Ours)	✓	✓	✓	✓		**Stateful Health AI**

Table 3. Prespecified mutation families.

ID	Seeded defect	Expected detector
MUT1	Remove stale-base check	P2
MUT2	Remove policy revalidation	P3, P15
MUT3	Remove consent revalidation	P4, P15
MUT4–6	Remove subject/actor/purpose equality checks	P5–P7
MUT7	Accept expired snapshot	P8
MUT8	Skip snapshot digest check	P8
MUT9	Allow THSS-consuming proposal to downgrade to unbound	P3–P8
MUT10	Break state/audit transaction atomicity	P13, P14
MUT11	Drop a provenance edge	P11, P14
MUT12	Reuse stale derived snapshot/cache	P4, P9, P12
MUT13	Select superseded assertion as current	P9, P10
MUT14	Break idempotency key semantics	P1
MUT15	Retry rejected proposal without renewed authorization	P15

fault families, not the intended final sample size. Where the implementation exposes multiple semantically distinct enforcement sites, the study instantiates multiple mutants per family. The target is at least 45 executable, non-duplicate mutants across several code locations; if the repository does not support that many defensible mutants, the final denominator and the resulting limitation are reported rather than padded with cosmetic edits.

Equivalent or unexecutable mutants are not silently removed. A prespecified exclusion rule requires a written rationale and raw evidence that the mutant cannot change externally observable behavior under the contract or cannot be built/executed. The denominator for mutation score is therefore auditable.

For the final freeze, non-equivalence screening used a blinded, frozen two-model review with Gemini 3.6 Flash High and Claude Sonnet 4.6. Each candidate was shown the invariant/fault definition, one-change diff, enforcement context, and reference-test semantics, and was labelled non-equivalent, equivalent, invalid, or uncertain before strategy outcomes were analyzed. Only mutually non-equivalent candidates (including those resolving to mutual non-equivalence after the prespecified reconciliation step) entered the final denominator. This is a reproducible dual-model screening procedure, not independent human expert adjudication; that distinction is retained as a validity limitation.

6 Evaluation Protocol

6.1 Primary endpoint

For strategy T, mutation score is

$$MS(T) = \frac{K_T}{N_{exec} - N_{eq}}, \tag{2}$$

where K_T is the number of non-equivalent executable mutants killed by T. The mutant, not each generated example, is the inferential object for this endpoint.

6.2 Secondary endpoints

For each killed mutant we record the first detecting invariant, wall-clock time to first counterexample, length of the minimal shrunk sequence, generated transitions, and seed. Across frozen seeds we report reproducibility (fraction of seeds killing the same mutant under the fixed budget), runtime cost, and flakiness. We also report an invariant-by-mutant kill matrix rather than a single aggregate score.

6.3 Budgets and freeze

The final statistics plan freezes framework version, seeds, maximum generated examples, maximum stateful steps, health-check settings, database backend, mutation manifest, and per-strategy time budget. Tuning of generator weights occurs only on development mutants excluded from the final set. A material code or manifest change after final result inspection requires a new run identifier.

7 Results

The sealed freeze `govmut-soict-2026-final-v2` used source revision `ab877e04`. Forty-five reviewed, non-equivalent mutants were evaluated across 16 frozen method/seed slots, yielding 720 executions with zero infrastructure errors. The unmutated baseline preflight passed all 16 method/seed checks, so no false kill was observed before mutation execution.

M3 detected every mutant killed by M0 and four additional mutants ($M0$-only $= 0$, $M3$-only $= 4$), so the observed improvement over regression alone was 8.9 percentage points but was not significant under the exact paired test ($p = 0.125$). In contrast, M3 detected 16 mutants missed by M1 and 14 missed by M2, with no reverse-only detections; both paired differences were significant. M0 itself detected more mutants than M1 ($p = 0.00183$) and M2 ($p = 0.0213$). Across seed-level executions the terminal ledger contained 161 KILLED, 559 SURVIVED, and zero INFRASTRUCTURE ERROR outcomes, but these 720 executions are not treated as an independent sample.

The result therefore does not support the simple claim that state-machine or stateless property testing alone is superior to an existing regression suite for this

Table 4. Sealed mutant-level strategy comparison. Seeds are execution streams, not independent experimental units; the denominator is 45 mutants for every strategy.

Strategy	Killed / 45	Mutation score (95% CI)	Exact paired comparison vs. M3
M0 Regression	16	0.356 (0.232–0.502)	$p = 0.125$
M1 Stateless properties	4	0.089 (0.035–0.207)	$p = 3.05 \times 10^{-5}$
M2 State machine	6	0.133 (0.063–0.262)	$p = 1.22 \times 10^{-4}$
M3 Combined	20	0.444 (0.309–0.588)	reference

corpus. M3 is the union of M0, M1, and M2, so its higher raw mutation score is not a budget-fair efficiency comparison. The defensible finding is complementarity: the combined suite preserved all regression detections and added four, while that incremental gain remains statistically uncertain. Equal-wall-clock or explicit cost-effectiveness analysis is required before making an efficiency claim.

Because these 45 mutants form a curated frozen corpus rather than a probability sample of future defects, the exact paired tests are secondary descriptive evidence. The primary engineering interpretation comes from the mutant-level kill pattern, unique detections by strategy, invariant/fault-family coverage, and runtime cost; no population-wide defect-frequency claim is attached to the p-values.

8 Interpretation

The mutation scores measure test adequacy on the frozen mutant corpus; they do not prove the absence of other defects. The combined strategy achieved the highest observed score, but its four-mutant advantage over regression alone was not statistically significant. The appropriate interpretation is therefore complementarity, not replacement.

The regression suite remained important: it killed 16 mutants and outperformed either stateless properties or state-machine testing alone on this frozen corpus. One plausible structural explanation is that regression tests directly exercise known enforcement guards, whereas generated state-machine traces spend a finite budget exploring a broader transition space; this observation should not be read as an intrinsic disadvantage of state-machine testing. The combined strategy preserved all regression detections and added four. Mutants that survived every strategy should be treated as evidence of residual test weakness or a need for further mutant review, not as proof that the seeded fault is harmless.

The result also illustrates why execution count is not the scientific sample size. The 720 method/seed executions improve reproducibility and expose variability, but the inferential unit for mutation score is the set of 45 reviewed mutants.

9 Threats to Validity

Construct validity. Mutation score is informative only when the seeded faults resemble meaningful implementation defects. The corpus targets health-information governance hazards such as accepting a stale medication-state write, continuing after consent or policy change, crossing subject/purpose boundaries, reusing an expired or tampered disclosure snapshot, dropping provenance, or breaking state/audit atomicity. We publish the mutation manifest, map every mutant to one or more contract invariants, and require an explicit rationale for equivalent or unexecutable exclusions. Equivalent-mutant classification remains a source of uncertainty, and the final classification used dual locked-model review rather than independent human expert adjudication.

Internal validity. Generated tests can produce different traces. Frozen seeds, fixed budgets, stored shrunk counterexamples, and repeated executions make this variation visible. Timing is hardware-dependent and is therefore secondary.

External validity. GLHS is the main system under test, and the 45-mutant corpus is modest compared with broad software mutation suites. The benchmark emphasizes semantic depth at real governance enforcement sites rather than scale. Results may not transfer to unrelated agent architectures or authorization models without further replication.

Clinical validity. This is a software-assurance study. It does not evaluate diagnosis, treatment recommendations, patient outcomes, or regulatory compliance.

10 Reproducibility

Artifacts are stored by immutable run ID and record Git revision, mutation-manifest hash, test-framework versions, seeds, state-machine budgets, backend configuration, machine metadata, exact counterexample traces, raw timings, and SHA-256 inventory. Generated manuscript tables are derived from the same sealed results. Missing runs remain NOT RUN. The complete machine-readable mutant-by-strategy outcome matrix, including per-seed execution metadata, is retained in `research/assurance_soict/results/final-analysis.json`.

11 Relationship to Prior GLHS Work

The GLHS paper studies a co-versioned disclosure-to-commit governance contract and reports its own model-context and concurrency evidence [15]. This paper asks a different question: how should such a stateful contract be tested, and which testing strategy detects seeded governance defects? The present study does not reuse the GLHS 64-subject model cohort as evidence and does not claim a new clinical or model-utility result.

12 Conclusion

GovMut-Health provides an executable mutation benchmark for sequence-sensitive governance faults in stateful health-AI systems. Its contribution is the health-specific fault taxonomy, invariant mapping, and controlled mutant-level comparison of established testing strategies. In the sealed 45-mutant experiment, the combined strategy achieved the highest observed mutation score (44.4%), but its four-mutant gain over regression alone was not significant; its gains over the stateless-property and state-machine strategies were significant. The evidence therefore supports using these techniques together on the tested governance-fault corpus. It does not establish formal verification or the absence of undiscovered defects.

References

1. Claessen, K., Hughes, J.: QuickCheck: A lightweight tool for random testing of Haskell programs. In: ICFP 2000, pp. 268–279. ACM (2000). https://doi.org/10.1145/351240.351266
2. Utting, M., Pretschner, A., Legeard, B.: A taxonomy of model-based testing approaches. Software Testing, Verification and Reliability 22(5), 297–312 (2012). https://doi.org/10.1002/stvr.456
3. Jia, Y., Harman, M.: An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering 37(5), 649–678 (2011). https://doi.org/10.1109/TSE.2010.62
4. Pretschner, A., Mouelhi, T., Le Traon, Y.: Model-Based Tests for Access Control Policies. In: ICST 2008, pp. 338–347. IEEE (2008). https://doi.org/10.1109/ICST.2008.44
5. Xu, D., Thomas, L., Kent, M., Mouelhi, T., Le Traon, Y.: A Model-Based Approach to Automated Testing of Access Control Policies. In: SACMAT 2012. ACM (2012). https://doi.org/10.1145/2295136.2295173
6. Hu, V.C., Kuhn, D.R., Yaga, D.: Verification and Test Methods for Access Control Policies/Models. NIST SP 800-192 (2017). https://doi.org/10.6028/NIST.SP.800-192
7. Li, X., Wang, Y., Lu, H., et al.: MemTX: Transactional Belief Commit for Stateful Agent Memory. arXiv:2607.23929 (2026)
8. Khan, S.: Verified Detection and Prevention of Concurrency Anomalies in Multi-Agent Large Language Model Systems. arXiv:2606.17182 (2026)
9. Sun, Y., Zhao, Y., Wang, Y., et al.: SWE-Mutation: Can LLMs Generate Reliable Test Suites in Software Engineering? arXiv:2605.22175 (2026)
10. Wang, Z.: TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv:2606.06240 (2026)
11. HL7 International: FHIR R4: Consent (2019), https://hl7.org/fhir/R4/consent.html
12. HL7 International: FHIR R4: Provenance (2019), https://hl7.org/fhir/R4/provenance.html
13. HL7 International: FHIR R4: RESTful API / HTTP (2019), https://hl7.org/fhir/R4/http.html
14. W3C: PROV-O: The PROV Ontology. W3C Recommendation (2013)
15. Thien, N.N., Quang, T.M.: GLHS: A Co-Versioned Disclosure-to-Commit Governance Contract for Longitudinal Health AI. Working preprint (2026)